

帧同步和状态同步

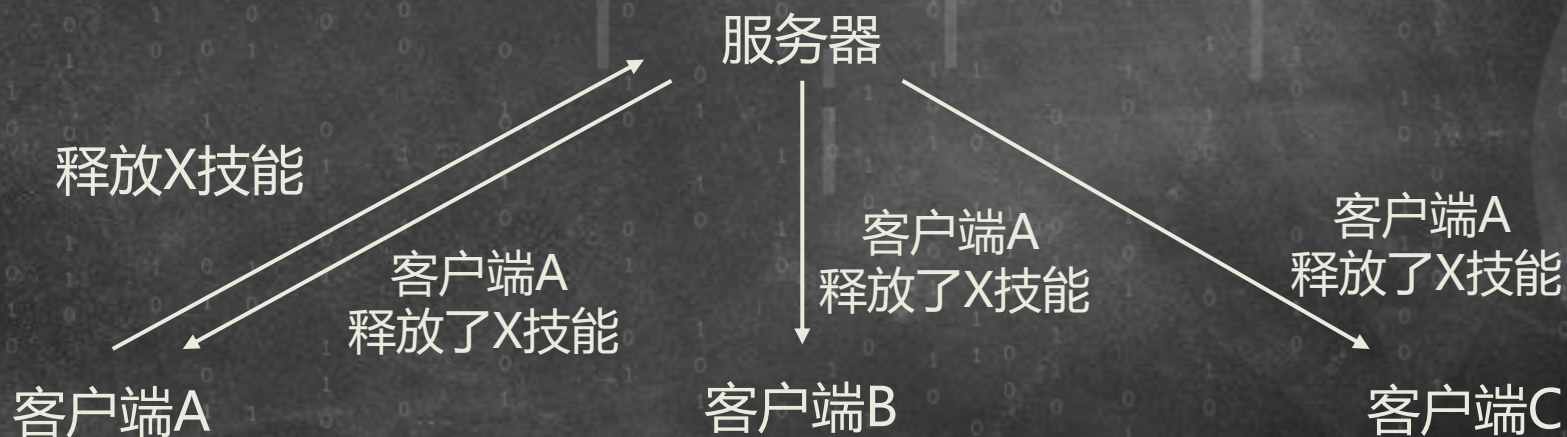


同步

- 所谓同步，就是多个客户端表现效果要一致，就好像很多个人一起跳舞，每个人的动作都要保持一致。
- 对于大多数游戏，不仅客户端的表现要一致，而且需要客户端和服务端的数据是一致的。所以，同步是一个网络游戏概念，只有网络游戏才需要同步，而单机游戏不需要同步。

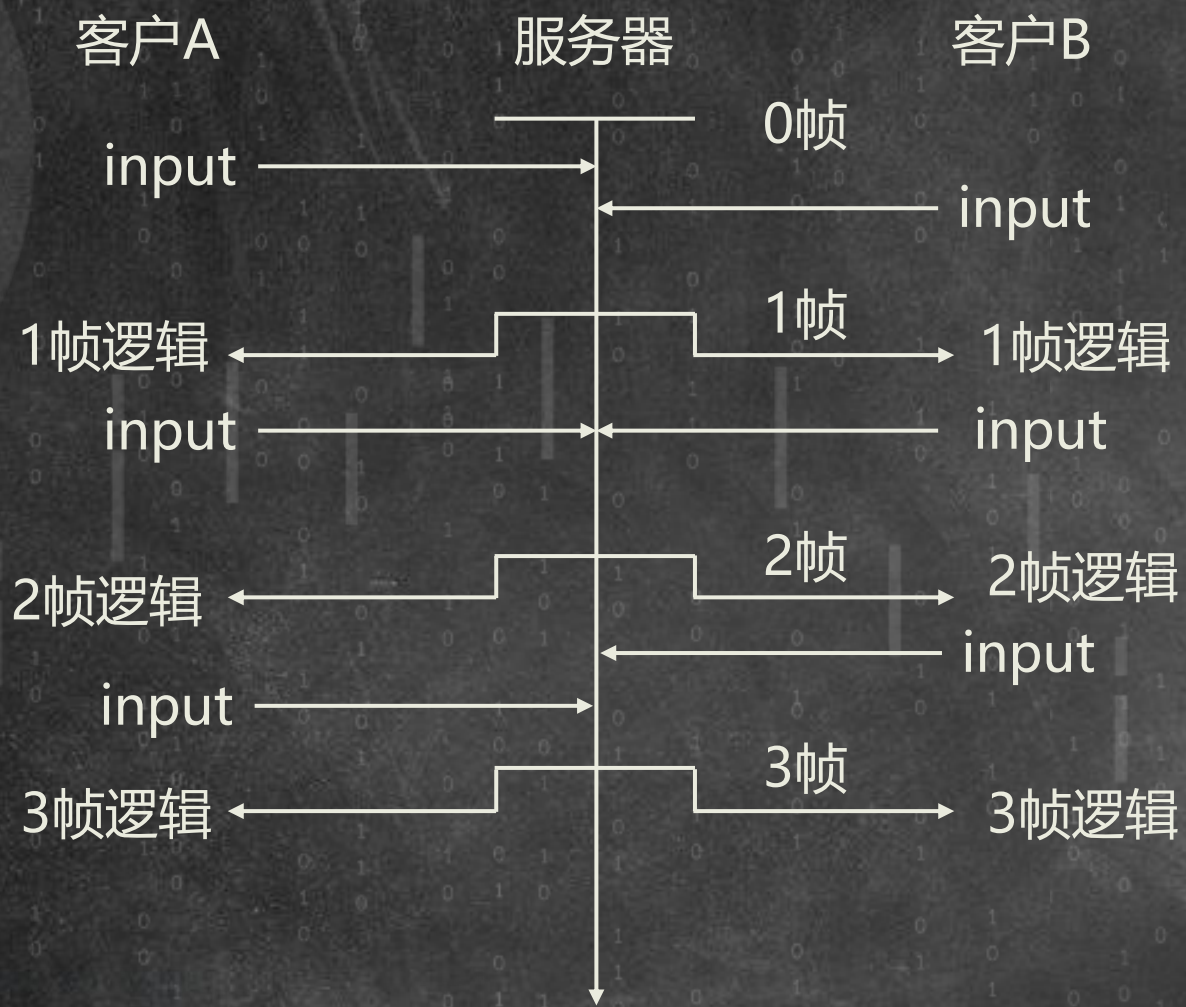
帧同步

- 帧同步的战斗逻辑在客户端。
- 战斗逻辑是包括技能逻辑、普攻、属性、伤害、移动、AI、检测、碰撞等等的一系列内容，这常常也被视为游戏开发过程中最难的部分。
- 帧同步下, 通信比较简单，服务端只转发操作，不做任何逻辑处理。
- 代表场景：MOBA游戏



帧同步流程

以老师收作业为例, 老师相当于服务器, 也有可能没有老师, 而是有个课代表既当学生也做收作业和发作业的工作 (这就是早期魔兽争霸和星际的局域网模式, 有个人充当主机), 学生可以在一天的任何时候提交作业, 老师会在每天的晚上12点, 清点一天的作业, 并把所有同学的作业 (电子版) 转发给所有其他所有同学让他们自己批阅打分 (如果老师自己批阅然后告诉同学们最终成绩就是状态同步)。所有同学和老师必须要有同样的打分标准, 也就是说同样的作业, 每个人批阅出来的成绩是必须要一样的 (不然导致游戏不同步)。



严格帧同步和锁帧

老师在晚上12点，必须清点所有同学的作业，确认收齐才可以。
不然，全班同学都得停下来等那些没交作业的同学，
除非那个同学退出这个班级，或者超过时间把这个学生开除掉。
这个就是为什么在玩魔兽争霸时，一人卡顿，全部人都卡主，然后那个人退了，
或者等一分钟还没连上就让那个人强制退出。
这个技术在现在来看是完全不合理的。

手感优化

- 游戏的网络情况比较复杂，丢包，延迟以及帧同步本身存在的延迟情况，导致优化画面或手感存在很大的问题，我们只能极大程度地弱化网络导致的画面卡顿情况。
- 可以通过预测帧完成
- 预测帧：以角色行走为例，由于网络卡顿，可能第一步执行完，第二三步延迟一会，一起过来，那么角色就会出现瞬移，采用预测帧的方式，可以让用户按照原来的方向，先走第二步，这样画面就会平滑，等第二步第三步一起来的时候，第二步由于执行了，就丢弃了，然后执行第三步。
- 帧校正 如果第二三步，是改变了方向的移动，那么就要校正他原来方向的第二步，直接校正会使画面一卡一卡的，这个相对复杂不在这里叙述。

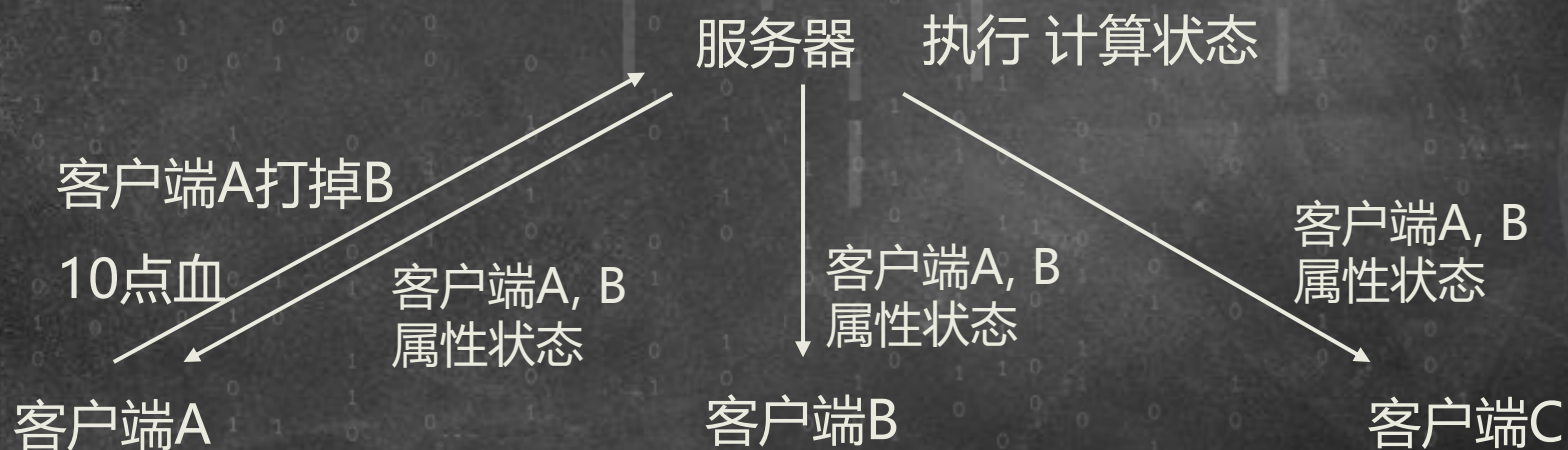
帧同步优缺点

- 优 • 开发方便，可以无视客户端服务器（但是要考虑逻辑和表现分离）
- 打击感反馈好，例如所有的命中都能在本地上马上触发相应的扣血和打击表现，反馈及时准确
- 网络流量小，带宽费用小、用户成本低、降低收发包带来的耗电
- 缺 • 对网络要求高。
- 反外挂能力差，容易本地修改，开图，修改属性等。
- 断线重连需要追帧
- 客户端逻辑计算性能压力大，需要每逻辑帧计算所有游戏单位的逻辑状态，即使单位不在屏幕内。
- 对结果一致性控制比较严格，如果使用了第三方的库，需要能够控制结果的一致性。

其他因素包括浮点计算在不同平台的误差差异、随机数序列一致性、一些容器的访问顺序问题。

状态同步

- 客户端更像一个服务端数据的表现层
- 一个英雄的几乎所有属性（例如血量、攻击、防御、攻速、魔法值等等）都是服务端传给客户端的，而且在属性发生改变的时候，服务端需要实时告诉客户端哪些属性改变了，客户端并不能改变这些属性，而是服务端传来多少属性就显示多少属性。
- 代表场景: MMO游戏



状态同步

优点

- 断线重连实现容易
- 防外挂实现容易

缺点

- 流量大
- 打击感反馈匹配不够精准，因为所有的表现都是服务器推送的，在网络波动、客户端服务器不同计算的误差下，客户端各个表现比较难契合。
- 对网络要求也会比较高，如果2中说的各个表现的契合，以及位移等都会受到网络波动的影响。

帧同步与状态同步的对比

	帧同步	状态同步
战斗核心逻辑	在客户端	在服务器
流量	小, 只需转发命令	大, 转发若干条属性, 并转发命令
回看和观战	容易 只需记录所有命令, 观战者再执行一遍	难 需要记录所有状态和命令, 然后按照时间执行一遍
安全性	会有外挂, 比如moba游戏全图挂, CS里面的透视挂, 锁头挂	更安全 所有数据都在服务器, 作弊需要攻击服务器
服务器压力		数据量更大, 压力更大, 且需要计算
开发效率	单机完成后, 做一些转发修改就可以完成网络版本	开发更难, 开发时不但要考虑客户端, 还有服务端, 要复杂很多
断线重连	需要把所有命令执行一遍	很简单, 只需要此时的所有状态

无论是状态同步还是帧同步，对于 网络延迟 的优化都会涉及到的问题是：

- 在网络协议层优化，可靠UDP代替TCP，降低延迟
- 在表现上优化，客户端先行、平滑插值等在表现上降低对延迟的感受

感谢大家的聆听

#####

